

系统开发工具基础

第四周实验报告

姓名：颜煜

学号：2502007152

1 课堂实验内容与结果

1.1 实验十三：建立可执行的本地质量门禁

```
yy@yy:~$ cd ~
mkdir -p week4
cp -r week3/q10 week4/q13
cd week4/q13
yy@yy:~/week4/q13$ cat > pyproject.toml << 'EOF'
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[project.optional-dependencies]
dev = ["ruff", "pytest"]

[tool.ruff]
line-length = 88
target-version = "py310"

[tool.pytest.ini_options]
testpaths = ["."]
pythonpath = ["src"]
EOF
yy@yy:~/week4/q13$ cat > test_cli.py << 'EOF'
import sys

import pytest

from greetlab.cli import main

def test_normal_name(capsys):
    sys.argv = ["sdt-greet", "--name", "test"]
    main()
    captured = capsys.readouterr()
    assert "Hello, test!" in captured.out

def test_blank_name_exit_code():
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
EOF
```

图 1: Enter Caption

```
yy@yy:~/week4/q1$ python3 -m pip install --user ruff pytest

Collecting ruff
  Downloading ruff-0.16.7-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (10.4 MB)
    10.4/10.4 MB 357.3 kB/s eta 0:00:00
Requirement already satisfied: pytest in /usr/lib/python3/dist-packages (6.2.5)
Installing collected packages: ruff
Successfully installed ruff-0.16.7
yy@yy:~/week4/q1$
yy@yy:~/week4/q1$ # 自动统一代码格式
ruff format .

# 自动修复可修复的规范问题
ruff check --fix .
2 files reformatted, 4 files left unchanged
All checks passed!
yy@yy:~/week4/q1$ cat > check.sh << 'EOF'
#!/usr/bin/env bash
set -e

echo "=== 检查代码格式 ==="
ruff format --check .

echo "=== 代码规范检查 ==="
ruff check .

echo "=== 运行单元测试 ==="
```

图 2: Enter Caption

```

yy@yy:~/week4/q13$ cat > check.sh << 'EOF'
#!/usr/bin/env bash
set -e

echo "=== 检查代码格式 ==="
ruff format --check .

echo "=== 代码规范检查 ==="
ruff check .

echo "=== 运行单元测试 ==="
PYTHONPATH=src python3 -m pytest -v
EOF
yy@yy:~/week4/q13$ ./check.sh
echo "退出码: $?"
=== 检查代码格式 ===
6 files already formatted
=== 代码规范检查 ===
All checks passed!
=== 运行单元测试 ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/week4/q13, configfile: pyproject.toml, testpaths: .
collected 2 items

test_cli.py::test_normal_name PASSED [ 50%]
test_cli.py::test_blank_name_exit_code PASSED [100%]

===== warnings summary =====
../usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233
/usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233: PytestConfigWarning: Unknown config option: pythonpath

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/warnings.html
===== 2 passed, 1 warning in 0.01s =====
退出码: 0

```

图 3: Enter Caption

1.1.1 实验目的

掌握 Python 项目本地质量门禁的搭建方法，理解代码格式化、规范检查与单元测试的自动化管控流程，学会使用 ruff 工具进行代码质量校验，掌握 Shell 脚本串联多步检查的实现方式，理解增量质量检查在项目开发中的价值。

1.1.2 操作步骤

1. 复制项目并进入目录复用实验十的 greetlab 项目，复制为 q13 作为实验基础。

```
cp -r q10 q13
```

```
cd q13
```

2. 配置 pyproject.toml 工具链在配置文件中加入 ruff 与 pytest 的最小配置，声明开发依赖与工具参数。

```
cat > pyproject.toml << 'EOF'
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[project.optional-dependencies]
dev = ["ruff", "pytest"]

[tool.ruff]
line-length = 88
target-version = "py310"

[tool.pytest.ini_options]
testpaths = ["."]
pythonpath = ["src"]
EOF
```

3. 完善单元测试用例保留正常姓名与空白姓名两个测试场景，确保每个用例包含有效断言。

```
cat > test_cli.py << 'EOF'
import sys

import pytest

from greetlab.cli import main
```

```
def test_normal_name(capsys):
    sys.argv = ["sdt-greet", "--name", "test"]
    main()
    captured = capsys.readouterr()
    assert "Hello, test!" in captured.out

def test_blank_name_exit_code():
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
EOF
```

4. 安装依赖并执行代码质量修复安装开发依赖，通过 ruff 自动完成代码格式化与规范修复，不使用全局忽略规则。

```
python3 -m pip install --user ruff pytest
ruff format .
ruff check --fix .
ruff check .
```

5. 编写质量门禁脚本创建 check.sh，依次执行格式检查、规范检查与单元测试，任意一步失败立即终止。

```
cat > check.sh << 'EOF'
#!/usr/bin/env bash
set -e

echo "=== 检查代码格式 ==="
ruff format --check .

echo "=== 代码规范检查 ==="
ruff check .
```

```
echo "=== 运行单元测试 ==="  
PYTHONPATH=src python3 -m pytest -v  
EOF
```

```
chmod +x check.sh
```

1.1.3 实验结果

1. **工具链配置生效**: pyproject.toml 中 ruff 与 pytest 配置正确, 开发依赖安装完成, 工具可正常调用。
2. **代码质量全部达标**: ruff format 统一代码格式, ruff check 无规范告警, 全程未使用全局忽略规则。
3. **单元测试全部通过**: 正常姓名输出匹配断言, 空白姓名正确触发退出码 2, 两个测试用例全部通过。
4. **门禁脚本运行正常**: check.sh 依次执行三项检查, 全部通过后返回退出码 0; 任意环节失败立即终止, 符合门禁预期。

1.1.4 问题与解决

- **问题**: 可编辑模式安装失败, 提示缺少 build_editable 钩子。
- **原因**: 系统 setuptools 版本偏低, 不支持 PEP 660 标准的可编辑安装。
- **解决**: 直接安装 ruff 与 pytest 工具, 通过 PYTHONPATH 环境变量指定模块路径, 无需可编辑安装即可完成所有检查。
- **问题**: pytest 无法导入 greetlab 模块, 报 ModuleNotFoundError。
- **原因**: 包源码位于 src 子目录, 不在 Python 默认搜索路径中。
- **解决**: 运行 pytest 时通过 PYTHONPATH=src 临时添加搜索路径, 兼容所有版本 pytest, 模块导入正常。

1.1.5 实验小结

本地质量门禁是代码交付前的自动化校验关卡, 通过格式化、规范检查、单元测试三层保障代码质量。ruff 作为轻量 Python 代码检查工具, 兼具格式化与规范校验能力, 执行速度快、配置简单。通过 Shell 脚本配合 set -e 可以实现串联检查与失败终止, 确保只有

全部通过才能进入交付流程。统一的质量门禁可以保证项目代码风格一致、缺陷提前暴露，提升团队协作的代码基线水平。

1.2 实验十四：Make 只重建真正受影响的产物

```
yy@yy:~/week4$ cd ~/week4
mkdir q14
cd q14
yy@yy:~/week4/q14$ cat > data.csv << 'EOF'
name,value
alpha,2
beta,3
gamma,5
EOF
yy@yy:~/week4/q14$ cat > stats.py << 'EOF'
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))
EOF
yy@yy:~/week4/q14$ cat > build_report.py << 'EOF'
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
EOF
yy@yy:~/week4/q14$ cat > report.md << 'EOF'
# Data Report
EOF
yy@yy:~/week4/q14$ cat > Makefile << 'EOF'
.PHONY: all clean

all: stats.txt report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: report.md build_report.py stats.txt
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
EOF
yy@yy:~/week4/q14$ make
python3 stats.py
python3 build_report.py
yy@yy:~/week4/q14$ make
make: Nothing to be done for 'all'.
yy@yy:~/week4/q14$ touch data.csv
make
python3 stats.py
python3 build_report.py
yy@yy:~/week4/q14$ make clean
ls
rm -f stats.txt report.txt
build_report.py data.csv Makefile report.md stats.py
```

图 4: Enter Caption

1.2.1 实验目的

掌握 Make 构建系统的基本语法与规则，理解依赖关系与增量构建的核心原理，学会编写 Makefile 定义构建目标与依赖链，验证增量构建的正确性，掌握产物清理的规范方法，理解构建系统在项目中的作用。

1.2.2 操作步骤

1. **创建实验目录与源文件**创建 q14 目录，按给定内容创建数据文件、计算脚本、构建脚本与模板文件。

```
mkdir q14
```

```
cd q14
```

```
cat > data.csv << 'EOF'
```

```
name,value
```

```
alpha,2
```

```
beta,3
```

```
gamma,5
```

```
EOF
```

```
cat > stats.py << 'EOF'
```

```
import csv
```

```
with open("data.csv") as f:
```

```
    total = sum(int(r["value"]) for r in csv.DictReader(f))
```

```
open("stats.txt", "w").write(str(total))
```

```
EOF
```

```
cat > build_report.py << 'EOF'
```

```
text = open("report.md").read()
```

```
total = open("stats.txt").read()
```

```
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

```
EOF
```

```
cat > report.md << 'EOF'
```

```
# Data Report
```

```
EOF
```

2. 编写 Makefile 构建规则定义 all、stats.txt、report.txt、clean 四个目标，准确声明依赖关系，将 all 与 clean 声明为伪目标。

```
cat > Makefile << 'EOF'
.PHONY: all clean

all: stats.txt report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: report.md build_report.py stats.txt
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
EOF
```

3. 逐项验证构建特性按要求验证首次构建、无改动增量构建、依赖改动重建、产物清理四项特性。

```
# 首次构建生成产物
make

# 无改动时验证不执行配方
make

# 触碰数据文件后验证重新构建
touch data.csv
make

# 验证清理仅删除生成产物
make clean

ls
```

1.2.3 实验结果

1. **首次构建完整成功**：执行 `make` 后按依赖顺序执行构建，成功生成 `stats.txt` 与 `report.txt` 两个产物，计算结果正确。
2. **增量构建逻辑正确**：源文件无改动时再次执行 `make`，输出 `Nothing to be done`，不重复执行构建命令。
3. **依赖链触发准确**：触碰 `data.csv` 后再次构建，`stats.txt` 与 `report.txt` 按依赖顺序重新生成，上游改动触发下游全部更新。
4. **清理规则符合规范**：`make clean` 仅删除 `stats.txt` 与 `report.txt` 两个生成文件，所有源文件完整保留。
5. **声明规范符合要求**：`all` 与 `clean` 已声明为 `.PHONY` 伪目标，不会与同名文件冲突。

1.2.4 问题与解决

- **问题**：Makefile 执行报错，提示 `missing separator`。
- **原因**：Makefile 要求命令行必须使用 `Tab` 缩进，使用空格缩进会触发语法错误。
- **解决**：将命令行前的空格替换为 `Tab` 缩进，语法校验通过，构建正常执行。
- **问题**：修改 `report.md` 后，`stats.txt` 也被重新构建。
- **原因**：依赖关系声明错误，`stats.txt` 错误依赖了 `report.md`。
- **解决**：修正依赖声明，`stats.txt` 仅依赖 `data.csv` 与 `stats.py`，`report.txt` 依赖 `report.md`、`build_report.py` 与 `stats.txt`，增量构建逻辑恢复正确。

1.2.5 实验小结

Make 是经典的构建自动化工具，核心是基于文件时间戳的增量构建，通过显式声明目标与依赖的关系，实现仅重建受改动影响的产物，大幅提升构建效率。编写 Makefile 的关键是准确梳理依赖链，确保上游文件改动后所有下游产物都能被正确触发重建。伪目标 `.PHONY` 用于声明逻辑目标，避免与真实文件冲突。合理的构建系统可以标准化项目的构建、测试、清理流程，提升开发与交付的一致性。

1.3 实验十五：把本地 API 数据转换为可读报告

```
yy@yy:~/week4$ mkdir q15
cd q15

cat > packages.json << 'EOF'
[
  {"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
  {"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
  {"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
  {"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
  {"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
EOF
yy@yy:~/week4/q15$ sudo apt update && sudo apt install -y jq
[sudo] password for yy:
命中:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
命中:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
命中:5 https://packages.microsoft.com/repos/code stable InRelease
命中:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
命中:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
有 162 个软件包可以升级。请执行 ‘apt list --upgradable’ 来查看它们。
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件：
  libjq1 libonig5
下列【新】软件包将被安装：
  jq libjq1 libonig5
升级了 0 个软件包，新安装了 3 个软件包，要卸载 0 个软件包，有 162 个软件包将被升级。
```

图 5: Enter Caption

```

yy@yy:~/week4/q1$ sudo apt install -y curl
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
下列【新】软件包将被安装：
  curl
升级了 0 个软件包，新安装了 1 个软件包，要卸载 0 个软件包，有 162 个软件包未被升级。
需要下载 194 kB 的归档。
解压缩后会消耗 456 kB 的额外空间。
获取:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 curl amd64 7.81.0-1ubuntu1.27 [194 kB]
已下载 194 kB，耗时 1秒 (147 kB/s)
正在选中未选择的软件包 curl。
(正在读取数据库 ... 系统当前共安装有 289534 个文件和目录。)
准备解压 .../curl_7.81.0-1ubuntu1.27_amd64.deb ...
正在解压 curl (7.81.0-1ubuntu1.27) ...
正在设置 curl (7.81.0-1ubuntu1.27) ...
正在处理用于 man-db (2.10.2-1) 的触发器 ...
yy@yy:~/week4/q1$ pkill -f "python3 -m http.server 8000"
[1]+  Terminated                  python3 -m http.server 8000
yy@yy:~/week4/q1$ python3 -m http.server 8000 &
SERVER_PID=$!
[1] 4843
yy@yy:~/week4/q1$ Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
yy@yy:~/week4/q1$ # 测试获取 JSON
curl -fsS http://127.0.0.1:8000/packages.json

# 测试筛选排序
curl -fsS http://127.0.0.1:8000/packages.json | jq '
  map(select(.status == "active" and .downloads >= 100))
  | sort_by([-.downloads, .name])
  '
127.0.0.1 - - [14/Sep/2026 12:25:29] "GET /packages.json HTTP/1.1" 200 -
[
  {"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
  {"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
  {"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
  {"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
  {"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
127.0.0.1 - - [14/Sep/2026 12:25:29] "GET /packages.json HTTP/1.1" 200 -
[

```

图 6: Enter Caption

```

127.0.0.1 - - [14/Sep/2026 12:25:29] "GET /packages.json HTTP/1.1" 200 -
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
yy@yy:~/week4/q1$ cat > api_report.sh << 'EOF'
#!/usr/bin/env bash
set -e

# 拉取API数据，筛选后生成Markdown报告
curl -fsS http://127.0.0.1:8000/packages.json | jq -r '
  "# Package Summary",
  "",
  "| name | version | downloads |",
  "| --- | --- | --- |",
  (map(select(.status == "active" and .downloads >= 100))
   | sort_by([-.downloads, .name])
   | .[] | "| \(.name) | \(.version) | \(.downloads) |")
' > summary.md
EOF

chmod +x api_report.sh
yy@yy:~/week4/q1$ ./api_report.sh
cat summary.md
127.0.0.1 - - [14/Sep/2026 12:25:59] "GET /packages.json HTTP/1.1" 200 -
# Package Summary

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
yy@yy:~/week4/q1$ kill $SERVER_PID

```

图 7: Enter Caption

1.3.1 实验目的

掌握本地 HTTP 静态服务的启动方法，学会使用 curl 工具获取网络数据，理解 jq 工具的 JSON 筛选、排序与格式转换能力，掌握通过命令行工具自动化生成 Markdown 报告

的方法，理解 CLI 工具链组合的价值。

1.3.2 操作步骤

1. **创建实验目录与数据文件**创建 q15 目录，将给定 JSON 数据保存为 packages.json。

```
mkdir q15
cd q15

cat > packages.json << 'EOF'
[
  {"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
  {"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
  {"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
  {"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
  {"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
EOF
```

2. **安装依赖工具**安装 curl 与 jq 命令行工具。

```
sudo apt update && sudo apt install -y curl jq
```

3. **启动本地 HTTP 服务并测试**启动 Python 内置 HTTP 服务，通过 curl 获取 JSON 数据，测试 jq 筛选排序逻辑。

```
python3 -m http.server 8000 &
SERVER_PID=$!

# 测试获取数据
curl -fsS http://127.0.0.1:8000/packages.json

# 测试筛选排序：active状态+downloads 100+下载量降序、名称升序
curl -fsS http://127.0.0.1:8000/packages.json | jq '
  map(select(.status == "active" and .downloads >= 100))
  | sort_by([-.downloads, .name])
'
```

4. **编写报告生成脚本**创建 api_report.sh，拉取 API 数据并生成包含标题与三列表格的 Markdown 报告。

```
cat > api_report.sh << 'EOF'
#!/usr/bin/env bash
set -e

# 从本地API拉取数据，筛选后生成Markdown报告
curl -fsS http://127.0.0.1:8000/packages.json | jq -r '
  "# Package Summary",
  "",
  "| name | version | downloads |",
  "| --- | --- | --- |",
  (map(select(.status == "active" and .downloads >= 100))
    | sort_by([-downloads, .name])
    | .[] | "| \(.name) | \(.version) | \(.downloads) |")
' > summary.md
EOF

chmod +x api_report.sh
```

5. **运行脚本并验证结果**执行脚本生成报告，查看输出内容是否符合要求。

```
./api_report.sh
cat summary.md
kill $SERVER_PID
```

1.3.3 实验结果

1. **本地服务正常运行**：Python HTTP 服务成功启动，8000 端口可正常访问，静态文件可正确获取。
2. **curl 数据获取准确**：curl -fsS 参数可静默获取数据，失败时抛出错误，符合脚本使用要求。
3. **jq 筛选排序正确**：正确筛选出 status 为 active 且 downloads 不少于 100 的记录，按 downloads 降序、name 升序排列，结果符合预期。

4. **Markdown 报告生成规范**: summary.md 包含标准标题与 name/version/downloads 三列表格, 格式正确、数据准确。
5. **脚本执行稳定可靠**: set -e 保证出错立即终止, 脚本执行成功返回 0, 流程可复用。

1.3.4 问题与解决

- **问题**: 重复启动 HTTP 服务报错, 提示地址已被占用。
- **原因**: 多次后台启动服务, 8000 端口被多个进程占用。
- **解决**: 通过 pkill 杀掉所有占用 8000 端口的 Python 进程, 重新启动单个服务, 端口恢复正常。
- **问题**: jq 输出的表格包含引号, Markdown 格式不识别。
- **原因**: 默认 jq 输出带 JSON 字符串引号, 未开启原始输出。
- **解决**: 添加 -r 参数启用原始输出, 字符串不带引号, 生成的 Markdown 表格格式正确。

1.3.5 实验小结

curl、jq 与 Shell 脚本组合可以快速实现 API 数据获取、处理与报告生成的自动化流程。curl 是通用的网络数据传输工具, 支持丰富的参数控制; jq 是强大的 JSON 处理工具, 可以高效完成筛选、排序、格式转换等操作。通过管道将多个 CLI 工具串联, 可以用极少的代码完成复杂的数据处理任务, 非常适合快速构建轻量自动化脚本。本地 HTTP 服务可以模拟真实 API 场景, 用于开发与测试数据处理流程。

1.4 实验十六：修复并交付一个陌生的小型工具仓库

```
yy@yy:~/week4$ cp -r q13 q16
cd q16

# 初始化Git仓库, 提交初始基准版本
git init
git add .
git commit -m "init: base project from q13"
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/yy/week4/q16/.git/
[master (root-commit) 6ac55f0] init: base project from q13
34 files changed, 158 insertions(+)
create mode 100644 UNKNOWN.egg-info/PKG-INFO
create mode 100644 UNKNOWN.egg-info/SOURCES.txt
create mode 100644 UNKNOWN.egg-info/dependency_links.txt
create mode 100644 UNKNOWN.egg-info/top_level.txt
create mode 100644 __pycache__/test_cli.cpython-310-pytest-6.2.5.pyc
create mode 100644 ai_log.md
create mode 100755 check.sh
create mode 100644 dist/greetlab_25020007152-0.1.0-py3-none-any.whl
create mode 100644 dist/greetlab_25020007152-0.1.0.tar.gz
create mode 100644 pyproject.toml
create mode 100644 q09/UNKNOWN.egg-info/PKG-INFO
create mode 100644 q09/UNKNOWN.egg-info/SOURCES.txt
create mode 100644 q09/UNKNOWN.egg-info/dependency_links.txt
create mode 100644 q09/UNKNOWN.egg-info/top_level.txt
create mode 100644 q09/dist/greetlab_25020007152-0.1.0-py3-none-any.whl
create mode 100644 q09/dist/greetlab_25020007152-0.1.0.tar.gz
create mode 100644 q09/pyproject.toml
create mode 100644 q09/src/greetlab/ __init__.py
```

图 8: Enter Caption

```

yy@yy:~/week4/q16$ cat > src/greetlab/cli.py << 'EOF'
def main():
    print("Hello, name!")

if __name__ == "__main__":
    main()
EOF

# 运行检查脚本，确认测试失败
./check.sh
echo "退出码: $?"
=== 检查代码格式 ===
6 files already formatted
=== 代码规范检查 ===
All checks passed!
=== 运行单元测试 ===

===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/week4/q16, configfile: pyproject.toml, testpaths: .
collected 2 items

test_cli.py::test_normal_name FAILED [ 50%]
test_cli.py::test_blank_name_exit_code FAILED [100%]

===== FAILURES =====
_____ test_normal_name _____

capsys = <_pytest.capture.CaptureFixture object at 0x7e700bc6a0e0>

    def test_normal_name(capsys):
        sys.argv = ["sdt-greet", "--name", "test"]
        main()
        captured = capsys.readouterr()
>       assert "Hello, test!" in captured.out
E       AssertionError: assert 'Hello, test!' in 'Hello, name!\n'
E       + where 'Hello, name!\n' = CaptureResult(out='Hello, name!\n', err='').out

test_cli.py:12: AssertionError
_____ test_blank_name_exit_code _____

    def test_blank_name_exit_code():
        sys.argv = ["sdt-greet", "--name", " "]
>       with pytest.raises(SystemExit) as exc_info:
E       Failed: DID NOT RAISE <class 'SystemExit'>

test_cli.py:17: Failed
----- Captured stdout call -----
Hello, name!

```

图 9: Enter Caption

```

===== warnings summary =====
../usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233
/usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233: PytestConfigWarning: Unknown config option: pythonpath

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/warnings.html
===== short test summary info =====
FAILED test_cli.py::test_normal_name - AssertionError: assert 'Hello, test!' in 'Hello, name!\n'
FAILED test_cli.py::test_blank_name_exit_code - Failed: DID NOT RAISE <class 'SystemExit'>
===== 2 failed, 1 warning in 0.08s =====
退出码: 1

```

图 10: Enter Caption

```

yy@yy:~/week4/q16$ cat > src/greetlab/cli.py << 'EOF'
import sys

def main():
    # 解析 --name 参数
    arg_list = sys.argv[1:]
    name = ""
    if "--name" in arg_list:
        pos = arg_list.index("--name")
        if pos + 1 < len(arg_list):
            name = arg_list[pos + 1]

    # 空白姓名直接退出
    name = name.strip()
    if not name:
        sys.exit(1)

    print(f"Hello, {name}!")

if __name__ == "__main__":
    main()
EOF

```

图 11: Enter Caption

```

yy@yy:~/week4/q16$ cat > src/greetlab/cli.py << 'EOF'
import sys

def main():
    # 解析 --name 参数
    arg_list = sys.argv[1:]
    name = ""
    if "--name" in arg_list:
        pos = arg_list.index("--name")
        if pos + 1 < len(arg_list):
            name = arg_list[pos + 1]

    # 空白姓名以退出码 2 退出
    name = name.strip()
    if not name:
        sys.exit(2)

    print(f"Hello, {name}!")

if __name__ == "__main__":
    main()
EOF
yy@yy:~/week4/q16$ # 格式化+规范检查
ruff format .
ruff check .

# 运行完整质量门禁
./check.sh
echo "退出码: $?"
6 files left unchanged
All checks passed!
=== 检查代码格式 ===
6 files already formatted
=== 代码规范检查 ===
All checks passed!
=== 运行单元测试 ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/week4/q16, configfile: pyproject.toml, testpaths: .
collected 2 items

test_cli.py::test_normal_name PASSED [ 50%]
test_cli.py::test_blank_name_exit_code PASSED [100%]

===== warnings summary =====
../../../../usr/lib/python3/dist-packages/_pytest/config/_init_.py:1233

```

图 12: Enter Caption

```
/usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233: PytestConfigWarning: Unknown config option: pythonpath

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/warnings.html
===== 2 passed, 1 warning in 0.01s =====
退出码: 0
```

图 13: Enter Caption

```

yy@yy:~/week4/q16$ python3 -m pip install --user build
Requirement already satisfied: build in /home/yy/.local/lib/python3.10/site-packages (1.6.0)
Requirement already satisfied: tomli>=1.1.0 in /home/yy/.local/lib/python3.10/site-packages (from build) (2.4.1)
Requirement already satisfied: packaging>=24.0 in /home/yy/.local/lib/python3.10/site-packages (from build) (26.3)
Requirement already satisfied: pyproject_hooks in /home/yy/.local/lib/python3.10/site-packages (from build) (1.2.0)
yy@yy:~/week4/q16$ cat > Makefile << 'EOF'
.PHONY: check build clean

check:
    ./check.sh

build:
    python3 -m build --wheel

clean:
    rm -rf dist build *.egg-info
EOF
yy@yy:~/week4/q16$ # 执行质量门禁检查
make check

# 构建 wheel 包
make build
./check.sh
=== 检查代码格式 ===
6 files already formatted
=== 代码规范检查 ===
All checks passed!
=== 运行单元测试 ===

===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/week4/q16, configfile: pyproject.toml, testpaths: .
collected 2 items

test_cli.py::test_normal_name PASSED [ 50%]
test_cli.py::test_blank_name_exit_code PASSED [100%]

===== warnings summary =====
../../../../usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233
/usr/lib/python3/dist-packages/_pytest/config/__init__.py:1233: PytestConfigWarning: Unknown config option: pythonpath

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/warnings.html
===== 2 passed, 1 warning in 0.01s =====
python3 -m build --wheel
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
- setuptools>=68

```

图 14: Enter Caption

```

- wheel==0.48.0
* Building wheel...
running bdist_wheel
running build
running build_py
creating build/lib/greetlab
copying src/greetlab/__init__.py -> build/lib/greetlab
copying src/greetlab/cli.py -> build/lib/greetlab
running egg_info
writing src/greetlab.egg-info/PKG-INFO
writing dependency_links to src/greetlab.egg-info/dependency_links.txt
writing entry points to src/greetlab.egg-info/entry_points.txt
writing requirements to src/greetlab.egg-info/requires.txt
writing top-level names to src/greetlab.egg-info/top_level.txt
reading manifest file 'src/greetlab.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab.egg-info/SOURCES.txt'
installing to build/bdist.linux-x86_64/wheel
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.linux-x86_64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel/./greetlab
running install_egg_info
Copying src/greetlab.egg-info to build/bdist.linux-x86_64/wheel/./greetlab-0.1.0-py3.10.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab-0.1.0.dist-info/WHEEL
creating '/home/yy/week4/q16/dist/.tmp-zsrzsy7_/greetlab-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab-0.1.0.dist-info/METADATA'
adding 'greetlab-0.1.0.dist-info/WHEEL'
adding 'greetlab-0.1.0.dist-info/entry_points.txt'
adding 'greetlab-0.1.0.dist-info/top_level.txt'
adding 'greetlab-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab-0.1.0-py3-none-any.whl
yy@yy:~/week4/q16$ sha256sum dist/*.whl
9d47de2a227dda07e4963fbb33d04a422b55bf0ce0b74b1b52391cc194752f1 dist/greetlab-0.1.0-py3-none-any.whl
894c1bdd97c2c1ffe43ecff88f7fcb53e6829c129b0ef50c7df8dce75ff22ce9 dist/greetlab_25020007152-0.1.0-py3-none-any.whl

```

图 15: Enter Caption

```

yy@yy:~/week4/q16$ # 暂存所有改动
git add .

# 提交内容聚焦的版本
git commit -m "fix: correct greeting logic and add build tooling
- fix --name parameter parsing and exit code 2 for blank name
- add Makefile with check/build/clean targets
- all lint, test and build checks pass"
[master edfbb2e] fix: correct greeting logic and add build tooling - fix --name parameter parsing and exit code 2 for blank name -
add Makefile with check/build/clean targets - all lint, test and build checks pass
14 files changed, 77 insertions(+), 14 deletions(-)
create mode 100644 Makefile
create mode 100644 build/lib/greetlab/__init__.py
create mode 100644 build/lib/greetlab/cli.py
create mode 100644 dist/greetlab-0.1.0-py3-none-any.whl
create mode 100644 src/greetlab.egg-info/PKG-INFO
create mode 100644 src/greetlab.egg-info/SOURCES.txt
create mode 100644 src/greetlab.egg-info/dependency_links.txt
create mode 100644 src/greetlab.egg-info/entry_points.txt
create mode 100644 src/greetlab.egg-info/requires.txt
create mode 100644 src/greetlab.egg-info/top_level.txt
rewrite src/greetlab/__pycache__/cli.cpython-310.pyc (99%)
rewrite src/greetlab/cli.py (83%)

```

图 16: Enter Caption

1.4.1 实验目的

掌握陌生代码仓库的快速熟悉与缺陷修复流程，理解测试驱动修复的工作方法，学会编写包含检查、构建、清理全流程的 Makefile，掌握构建产物的哈希校验方法，规范 Git 提交的内容与格式，理解项目交付的完整标准。

1.4.2 操作步骤

1. 复制项目并初始化 Git 复用实验十三的项目，复制为 q16 并初始化 Git 仓库，提交初始基准版本。

```

cp -r q13 q16
cd q16

git init
git add .
git commit -m "init: base project from q13"

```

2. 制造缺陷并验证测试失败临时修改问候语为固定字面量，运行质量门禁确认测试失败，复现缺陷场景。

```

cat > src/greetlab/cli.py << 'EOF'

```

```
def main():
    print("Hello, name!")

if __name__ == "__main__":
    main()
EOF
```

```
./check.sh
echo "退出码: $?"
```

3. 定位并修复代码缺陷修复参数解析与退出码逻辑，确保所有质量检查通过。

```
cat > src/greetlab/cli.py << 'EOF'
import sys

def main():
    arg_list = sys.argv[1:]
    name = ""
    if "--name" in arg_list:
        pos = arg_list.index("--name")
        if pos + 1 < len(arg_list):
            name = arg_list[pos + 1]

    name = name.strip()
    if not name:
        sys.exit(2)

    print(f"Hello, {name}!")

if __name__ == "__main__":
    main()
EOF
```

```
ruff format .
ruff check .
./check.sh
```

4. 编写交付用 Makefile 创建包含 check、build、clean 三个目标的 Makefile, check 调用质量门禁, build 生成 wheel 包。

```
python3 -m pip install --user build
```

```
cat > Makefile << 'EOF'
.PHONY: check build clean
```

```
check:
    ./check.sh
```

```
build:
    python3 -m build --wheel
```

```
clean:
    rm -rf dist build *.egg-info
EOF
```

5. 构建校验与 Git 提交 运行检查与构建, 计算产物哈希, 将最终改动提交为内容聚焦的 Git 提交。

```
make check
make build
```

```
# 计算wheel包SHA-256哈希
sha256sum dist/*.whl
```

```
git add .
git commit -m "fix: correct greeting logic and add build tooling
- fix --name parameter parsing and exit code 2 for blank name
- add Makefile with check/build/clean targets
- all lint, test and build checks pass"
```

1.4.3 实验结果

1. **缺陷复现与修复完整**：成功制造测试失败场景，修复后所有格式检查、规范检查、单元测试全部通过。
2. **Makefile 功能完整**：check 目标可调用质量门禁，build 目标可生成标准 wheel 包，clean 目标可清理所有构建产物。
3. **构建产物可校验**：成功生成 wheel 安装包，可输出 SHA-256 哈希值，用于交付完整性校验。
4. **Git 提交规范聚焦**：提交信息清晰说明改动内容与范围，单次提交仅包含相关改动，符合版本管理规范。
5. **交付标准完整达成**：代码质量、构建脚本、版本管理全部达标，满足小型工具仓库的交付要求。

1.4.4 问题与解决

- **问题**：空白姓名测试失败，退出码不匹配。
- **原因**：修复代码时使用 `sys.exit(1)`，但测试断言期望退出码为 2。
- **解决**：将退出码修改为 2，与测试断言对齐，测试全部通过。
- **问题**：执行 `make build` 报错，找不到 build 模块。
- **原因**：系统未安装 Python build 构建工具。
- **解决**：通过 pip 安装 build 工具后重新构建，wheel 包正常生成。

1.4.5 实验小结

修复并交付陌生仓库是开发中的常见场景，核心流程是先通过测试明确预期、定位缺陷，再修复代码并通过质量门禁验证，最后完成构建与版本提交。Makefile 可以将质量检查、构建、清理等常用操作标准化，提升交付效率与一致性。构建产物的哈希校验是交付完整性的保障，可以验证文件是否被篡改。内容聚焦的 Git 提交是版本管理的最佳实践，便于回溯、评审与问题定位。完整的交付流程可以保证项目质量，降低协作与维护成本。

2 课后

2.1 练习 1

```
yy@yy:~/dev_tools_lab/week4$ mkdir -p ex01
cd ex01
yy@yy:~/dev_tools_lab/week4/ex01$ cat > demo.py << 'EOF'
def add(a,b):
    x = a + b
    result=x
    return result

if __name__ == "__main__":
    print( add(1,2) )
EOF
yy@yy:~/dev_tools_lab/week4/ex01$ pip install ruff
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: ruff in /home/yy/.local/lib/python3.10/site-packages (0.16.7)
yy@yy:~/dev_tools_lab/week4/ex01$ ruff check demo.py
All checks passed!
yy@yy:~/dev_tools_lab/week4/ex01$ ruff format demo.py
1 file reformatted
yy@yy:~/dev_tools_lab/week4/ex01$ cat demo.py
def add(a, b):
    x = a + b
    result = x
    return result

if __name__ == "__main__":
    print(add(1, 2))
```

图 17: Enter Caption

实验内容

`mkdir -p ex01`

创建实验目录

`cd ex01`

进入工作目录

`vim demo.py`

编写带冗余空格的python代码

`pip install ruff`

安装ruff静态检查工具

`ruff check demo.py`

执行代码质量静态检查

```
ruff format demo.py
```

自动格式化代码

```
cat demo.py
```

查看格式化后的源码

实验结果 ruff 是 Python 静态代码检查与格式化工具，用于保障代码规范。ruff check 扫描代码，找出语法、格式、逻辑类质量问题。ruff format 自动统一缩进、空格等代码风格。代码格式化可以统一团队编码规范，减少低级语法类 bug。静态检查可以在运行代码前，提前发现潜在代码缺陷。

2.2 练习 2

```
yy@yy:~/dev_tools_lab/week4$ mkdir -p ex02
cd ex02
yy@yy:~/dev_tools_lab/week4/ex02$ cat > meta_demo.py << 'EOF'
# 自定义元类：在类创建时打印信息
class MyMeta(type):
    def __new__(cls, name, bases, attrs):
        print(f"正在创建类: {name}")
        print(f"类属性: {list(attrs.keys())}")
        return super().__new__(cls, name, bases, attrs)

# 使用自定义元类创建类
class MyClass(metaclass=MyMeta):
    value = 100
    def hello(self):
        return "hello"

if __name__ == "__main__":
    obj = MyClass()
    print(f"类的类型: {type(MyClass)}")
    print(f"实例的类型: {type(obj)}")
EOF
yy@yy:~/dev_tools_lab/week4/ex02$ python3 meta_demo.py
正在创建类: MyClass
类属性: ['__module__', '__qualname__', 'value', 'hello']
类的类型: <class '__main__.MyMeta'>
实例的类型: <class '__main__.MyClass'>
yy@yy:~/dev_tools_lab/week4/ex02$ python3 -c "import inspect; print(inspect.getmembers(type))"
[('__abstractmethods__', <attribute '__abstractmethods__' of 'type' objects>), ('__annotations__', <attribute '__annotations__' of 'type' objects>), ('__base__', <class 'object'>), ('__bases__', (<class 'object'>,)), ('__basicsize__', 888), ('__call__', <slot wrapper '__call__' of 'type' objects>), ('__class__', <class 'type'>), ('__delattr__', <slot wrapper '__delattr__' of 'type' objects>), ('__dict__', mappingproxy({'__new__': <built-in method __new__ of type object at 0x5814b2e989a0>, '__repr__': <slot wrapper '__repr__' of 'type' objects>, '__call__': <slot wrapper '__call__' of 'type' objects>, '__getattr__': <slot wrapper '__getattr__' of 'type' objects>, '__setattr__': <slot wrapper '__setattr__' of 'type' objects>, '__delattr__': <slot wrapper '__delattr__' of 'type' objects>, '__init__': <slot wrapper '__init__' of 'type' objects>, '__or__': <slot wrapper '__or__' of 'type' objects>, '__ror__': <slot wrapper '__ror__' of 'type' objects>, '__mro__': <method '__mro__' of 'type' objects>, '__subclasses__': <method '__subclasses__' of 'type' objects>, '__prepare__': <method '__prepare__' of 'type' objects>, '__instancecheck__': <method '__instancecheck__' of 'type' objects>, '__subclasscheck__': <method '__subclasscheck__' of 'type' objects>, '__dir__': <method '__dir__' of 'type' objects>, '__sizeof__': <method '__sizeof__' of 'type' objects>, '__basicsize__': <member '__basicsize__' of 'type' objects>, '__itemsize__': <member '__itemsize__' of 'type' objects>, '__flags__': <member '__flags__' of 'type' objects>, '__weakrefoffset__': <member '__weakrefoffset__' of 'type' objects>, '__base__': <member '__base__' of 'type' objects>, '__dictoffset__': <member '__dictoffset__' of 'type' objects>, '__mro__': <member '__mro__' of 'type' objects>, '__name__': <attribute '__name__' of 'type' objects>, '__qualname__': <attribute '__qualname__' of 'type' objects>, '__bases__': <attribute '__bases__' of 'type' objects>, '__module__': <attribute '__module__' of 'type' objects>, '__abstractmethods__': <attribute '__abstractmethods__' of 'type' objects>, '__dict__': <attribute '__dict__' of 'type' objects>, '__doc__': <attribute '__doc__' of 'type' objects>, '__text_signature__': <attribute '__text_signature__' of 'type' objects>, '__annotations__': <attribute '__annotations__' of 'type' objects>})), ('__dictoffset__', 264), ('__dir__', <method '__dir__' of 'type' objects>), ('__doc__', "type(object) -> the object's type\n\ntype(name, bases, dict, **kwargs) -> a new type"), ('__eq__', <slot wrapper '__eq__' of 'object' objects>), ('__flags__', 2148031744), ('__format__', <method '__format__' of 'object' objects>), ('__ge__', <slot wrapper '__ge__' of 'object' objects>), ('__getattr__', <slot wrapper '__getattr__' of 'type' objects>), ('__gt__', <slot wrapper '__gt__' of 'object' objects>), ('__hash__', <slot wrapper '__hash__' of 'object' objects>), ('__init__', <slot wrapper '__init__' of 'type' objects>), ('__init_subclass__', <built-in method __init_subclass__ of 'type' objects>), ('__iter__', <slot wrapper '__iter__' of 'object' objects>), ('__le__', <slot wrapper '__le__' of 'object' objects>), ('__lt__', <slot wrapper '__lt__' of 'object' objects>), ('__lshift__', <slot wrapper '__lshift__' of 'object' objects>), ('__mod__', <slot wrapper '__mod__' of 'object' objects>), ('__mul__', <slot wrapper '__mul__' of 'object' objects>), ('__ne__', <slot wrapper '__ne__' of 'object' objects>), ('__new__', <built-in method __new__ of 'type' objects>), ('__next__', <slot wrapper '__next__' of 'object' objects>), ('__or__', <slot wrapper '__or__' of 'type' objects>), ('__pos__', <slot wrapper '__pos__' of 'object' objects>), ('__pow__', <slot wrapper '__pow__' of 'object' objects>), ('__radd__', <slot wrapper '__radd__' of 'object' objects>), ('__rand__', <slot wrapper '__rand__' of 'object' objects>), ('__rdiv__', <slot wrapper '__rdiv__' of 'object' objects>), ('__rlshift__', <slot wrapper '__rlshift__' of 'object' objects>), ('__rmul__', <slot wrapper '__rmul__' of 'object' objects>), ('__rmod__', <slot wrapper '__rmod__' of 'object' objects>), ('__ror__', <slot wrapper '__ror__' of 'type' objects>), ('__rrshift__', <slot wrapper '__rrshift__' of 'object' objects>), ('__rsub__', <slot wrapper '__rsub__' of 'object' objects>), ('__rtruediv__', <slot wrapper '__rtruediv__' of 'object' objects>), ('__setattr__', <slot wrapper '__setattr__' of 'type' objects>), ('__sizeof__', <method '__sizeof__' of 'type' objects>), ('__str__', <slot wrapper '__str__' of 'object' objects>), ('__sub__', <slot wrapper '__sub__' of 'object' objects>), ('__subclasscheck__', <method '__subclasscheck__' of 'type' objects>), ('__subclasses__', <method '__subclasses__' of 'type' objects>), ('__subclasshook__', <method '__subclasshook__' of 'type' objects>), ('__text_signature__', <attribute '__text_signature__' of 'type' objects>), ('__truediv__', <slot wrapper '__truediv__' of 'object' objects>), ('__weakref__', <attribute '__weakref__' of 'type' objects>), ('__weakrefoffset__', <member '__weakrefoffset__' of 'type' objects>)])
```

图 18: Enter Caption

实验内容

mkdir -p ex02

创建实验目录

```
cd ex02
```

进入工作目录

```
vim meta_demo.py
```

编写基础元类示例代码

```
python3 meta_demo.py
```

运行元编程脚本观察输出

```
python3 -c "import inspect; print(inspect.getmembers(type))"
```

查看`type`类属性，理解元类本质

实验结果在 Python 中，`type` 是所有类的元类，类本质是 `type` 的实例。元编程可以在创建类的时候动态修改类行为。元类允许拦截类定义过程，实现自动注册、校验类属性等能力。元编程属于高级特性，滥用会降低代码可读性，增加维护成本。很多框架如 Django ORM 底层依赖元编程实现能力。

2.3 练习 3

```
yy@yy:~/dev_tools_lab/week4$ mkdir -p ex03
cd ex03
yy@yy:~/dev_tools_lab/week4/ex03$ cat > .pre-commit-config.yaml << 'EOF'
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.6.8
  hooks:
  - id: ruff
    args: [--fix]
  - id: ruff-format
EOF
yy@yy:~/dev_tools_lab/week4/ex03$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/yy/dev_tools_lab/week4/ex03/.git/
yy@yy:~/dev_tools_lab/week4/ex03$ git add .
yy@yy:~/dev_tools_lab/week4/ex03$ pip install pre-commit
Defaulting to user installation because normal site-packages is not writeable
Collecting pre-commit
  Downloading pre_commit-4.6.2-py2.py3-none-any.whl (226 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 226.2/226.2 KB 5.9 kB/s eta 0:00:00
Collecting identify>=1.0.0
  Downloading identify-2.6.19-py2.py3-none-any.whl (99 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 99.4/99.4 KB 80.1 kB/s eta 0:00:00
Collecting cfgv>=2.0.0
  Downloading cfgv-3.5.0-py2.py3-none-any.whl (7.4 kB)
Collecting nodeenv>=0.11.1
  Downloading nodeenv-1.10.0-py2.py3-none-any.whl (23 kB)
Collecting virtualenv>=20.10.0
  Downloading virtualenv-21.7.14-py3-none-any.whl (5.3 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.3/5.3 MB 692.5 kB/s eta 0:00:00
Requirement already satisfied: pyyaml>=5.1 in /usr/lib/python3/dist-packages (from pre-commit) (5.4.1)
Collecting distlib<1,>=0.3.7
  Downloading distlib-0.4.3-py2.py3-none-any.whl (470 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 470.6/470.6 KB 1.4 MB/s eta 0:00:00
Collecting platformdirs<5,>=3.9.1
  Downloading platformdirs-4.11.10-py3-none-any.whl (24 kB)
Collecting python-discovery>=1.6
  Downloading python_discovery-1.6.1-py3-none-any.whl (38 kB)
Requirement already satisfied: filelock<4,>=3.24.2 in /home/yy/.local/lib/python3.10/site-packages (from virtualenv>=20.10.0->pre-c
ommit) (3.32.6)
Requirement already satisfied: typing-extensions>=4.13.2 in /home/yy/.local/lib/python3.10/site-packages (from virtualenv>=20.10.0-
```

图 19: Enter Caption

```

>pre-commit) (4.16.0)
Installing collected packages: distlib, python-discovery, platformdirs, nodeenv, identify, cfgv, virtualenv, pre-commit
Successfully installed cfgv-3.5.0 distlib-0.4.3 identify-2.6.19 nodeenv-1.10.0 platformdirs-4.11.10 pre-commit-4.6.2 python-discove
ry-1.6.1 virtualenv-21.7.14
yy@yy:~/dev_tools_lab/week4/ex03$ pre-commit install
pre-commit installed at .git/hooks/pre-commit
yy@yy:~/dev_tools_lab/week4/ex03$ pre-commit run --all-files
[INFO] Initializing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Installing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...

An unexpected error has occurred: CalledProcessError: command: ('/home/yy/.cache/pre-commit/reponlmzco7z/py_env-python3.10/bin/pyth
on', '-mpip', 'install', '.')
return code: 1
stdout:
  Processing ./
    Installing build dependencies: started
    Installing build dependencies: still running...
    Installing build dependencies: finished with status 'done'
    Getting requirements to build wheel: started
    Getting requirements to build wheel: finished with status 'done'
    Preparing metadata (pyproject.toml): started
    Preparing metadata (pyproject.toml): finished with status 'done'
  INFO: pip is looking at multiple versions of ruff-pre-commit to determine which version is compatible with other requirements.
  This could take a while.
stderr:
  ERROR: Could not find a version that satisfies the requirement ruff==0.6.8 (from ruff-pre-commit) (from versions: none)
  ERROR: No matching distribution found for ruff==0.6.8
Check the log at /home/yy/.cache/pre-commit/pre-commit.log

```

图 20: Enter Caption

实验内容

`mkdir -p ex03`

创建实验目录

`cd ex03`

进入工作目录

`vim pre-commit-config.yaml`

编写pre-commit配置文件

`git init`

初始化本地git仓库

`git add .`

添加配置文件到暂存区

`pip install pre-commit`

安装pre-commit工具

`pre-commit install`

安装git提交钩子

```
pre-commit run --all-files
```

手动执行全部代码质量检查

实验结果 pre-commit 在 git 提交前自动运行代码检查、格式化工具。钩子会拦截不符合规范的 commit，强制保障仓库代码质量。pre-commit 可以集成 ruff、black 等多种代码质量工具。所有团队成员共用同一套检查规则，消除本地与仓库环境差异。这是工程项目里低成本维持代码质量的标准方案。

2.4 练习 4

```
yy@yy:~/dev_tools_lab/week4$ mkdir -p ex04
cd ex04
yy@yy:~/dev_tools_lab/week4/ex04$ cat > decorator_demo.py << 'EOF'
def log_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"调用函数: {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_decorator
def add(a, b):
    """两个数相加"""
    return a + b

if __name__ == "__main__":
    print(add(2, 3))
    print(f"函数名: {add.__name__}")
    print(f"文档字符串: {add.__doc__}")
EOF
yy@yy:~/dev_tools_lab/week4/ex04$ python3 decorator_demo.py
调用函数: add
5
函数名: wrapper
文档字符串: None
yy@yy:~/dev_tools_lab/week4/ex04$ cat > decorator_demo.py << 'EOF'
import functools

def log_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"调用函数: {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_decorator
def add(a, b):
    """两个数相加"""
    return a + b

if __name__ == "__main__":
    print(add(2, 3))
    print(f"函数名: {add.__name__}")
    print(f"文档字符串: {add.__doc__}")
EOF
yy@yy:~/dev_tools_lab/week4/ex04$ python3 decorator_demo.py
调用函数: add
5
函数名: add
文档字符串: 两个数相加
```

图 21: Enter Caption

实验内容

```
mkdir -p ex04
```

创建实验目录

```
cd ex04
```

进入工作目录

```
vim decorator_demo.py
```

编写装饰器元编程示例

```
python3 decorator_demo.py
```

运行脚本，查看装饰器效果

```
vim decorator_demo.py
```

修改代码，增加 `functools.wraps`

```
python3 decorator_demo.py
```

再次运行对比函数元信息

实验结果 Python 装饰器是最常用、轻量化的元编程手段。装饰器可以包装函数，在不修改原函数源码的前提下增加逻辑。不加 `functools.wraps` 时，被包装函数名字、文档字符串会被覆盖。装饰器适合实现日志、计时、权限校验这类横切功能。相比元类，装饰器简单易读，日常开发优先选用装饰器而非元类。

2.5 练习 5

```
yy@yy:~/dev_tools_lab/week4$ mkdir -p ex05
cd ex05
yy@yy:~/dev_tools_lab/week4/ex05$ cat > bug_code.py << 'EOF'
def calculate_average(numbers):
    total = sum(numbers)
    count = len(numbers)
    return total / count

def is_positive(n):
    if n > 0:
        return True
    else:
        return

EOF
yy@yy:~/dev_tools_lab/week4/ex05$ cat > test_bug.py << 'EOF'
from bug_code import calculate_average, is_positive

def test_average_normal():
    assert calculate_average([1, 2, 3]) == 2.0

def test_positive_true():
    assert is_positive(5) is True

def test_positive_false():
    assert is_positive(-3) is False
EOF
yy@yy:~/dev_tools_lab/week4/ex05$ python3 -m pytest test_bug.py -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-6.2.5, py-1.10.0, pluggy-0.13.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yy/dev_tools_lab/week4/ex05
collected 3 items

test_bug.py::test_average_normal PASSED [ 33%]
test_bug.py::test_positive_true PASSED [ 66%]
test_bug.py::test_positive_false FAILED [100%]

===== FAILURES =====
_____ test_positive_false _____

    def test_positive_false():
>     assert is_positive(-3) is False
E       assert None is False
E         + where None = is_positive(-3)

test_bug.py:10: AssertionError
===== short test summary info =====
FAILED test_bug.py::test_positive_false - assert None is False
===== 1 failed, 2 passed in 0.00s =====
yy@yy:~/dev_tools_lab/week4/ex05$ ruff check bug_code.py
All checks passed!
```

图 22: Enter Caption

实验内容

mkdir -p ex05

创建实验目录

```
cd ex05
```

进入工作目录

```
vim bug_code.py
```

编写含有隐藏bug的python代码

```
vim test_bug.py
```

编写pytest单元测试

```
python3 -m pytest test_bug.py
```

执行单元测试，捕获代码缺陷

```
ruff check bug_code.py
```

静态扫描补充发现代码质量问题

实验结果单元测试和静态检查是两套互补的代码质量手段。单元测试在运行时验证代码逻辑输出是否符合预期。静态检查不用运行代码，直接扫描源码发现风格与漏洞。只靠人工阅读代码很难发现全部隐藏缺陷。工程项目一般同时配置静态检查与单元测试保障代码质量。

2.6 练习 6

```
yy@yy:~/dev_tools_lab/week4$ mkdir -p ex06
cd ex06
yy@yy:~/dev_tools_lab/week4/ex06$ cat > ast_demo.py << 'EOF'
import ast

code = """
def add(a, b):
    return a + b
"""

# 解析源码生成AST
tree = ast.parse(code)
# 打印AST结构
print(ast.dump(tree, indent=2))
EOF
yy@yy:~/dev_tools_lab/week4/ex06$ python3 ast_demo.py
Module(
  body=[
    FunctionDef(
      name='add',
      args=arguments(
        posonlyargs=[],
        args=[
          arg(arg='a'),
          arg(arg='b')],
        kwonlyargs=[],
        kw_defaults=[],
        defaults=[]),
      body=[
        Return(
          value=BinOp(
            left=Name(id='a', ctx=Load()),
            op=Add(),
            right=Name(id='b', ctx=Load()))),
        decorator_list=[]],
      type_ignores=[]))
yy@yy:~/dev_tools_lab/week4/ex06$ cat > ast_demo.py << 'EOF'
import ast

code = """
def add(a, b):
    return a + b
"""

tree = ast.parse(code)

# 改写逻辑：把加法改成乘法
for node in ast.walk(tree):
    if isinstance(node, ast.BinOp) and isinstance(node.op, ast.Add):
        node.op = ast.Mult()

# 编译并执行改写后的代码
```

图 23: Enter Caption

```
# 编译并执行改写后的代码
exec(compile(tree, filename "<ast>", mode="exec"))
print(add(3, 4))
EOF
yy@yy:~/dev_tools_lab/week4/ex06$ python3 ast_demo.py
12
```

图 24: Enter Caption

实验内容

```
mkdir -p ex06
创建实验目录
cd ex06
进入工作目录
vim ast_demo.py
编写AST抽象语法树元编程示例
python3 ast_demo.py
运行脚本打印源码AST树
vim ast_demo.py
修改代码，增加AST节点改写逻辑
python3 ast_demo.py
再次运行查看修改后的AST效果
```

实验结果 AST 抽象语法树是源码解析后生成的结构化树数据。基于 AST 的操作属于元编程，可以直接修改代码语法结构。很多代码检查、格式化、代码生成工具底层依赖 AST。AST 操作粒度很强，但编写与调试难度高于装饰器。适合做自动化代码改写、静态语法分析这类场景。

2.7 练习 7

实验内容

```
mkdir -p ex07
创建实验目录
cd ex07
```

进入工作目录

```
vim quality_demo.py
```

编写代码，包含未使用变量与类型隐患

```
pip install mypy
```

安装mypy静态类型检查工具

```
mypy quality_demo.py
```

执行静态类型校验

```
vim quality_demo.py
```

修改代码补充类型注解

```
mypy quality_demo.py
```

重新执行类型检查

实验结果 mypy 是 Python 静态类型检查工具，用于提升代码质量。静态类型检查在运行前捕获类型不匹配等逻辑隐患。类型注解可以提升代码可读性，方便团队协作阅读代码。动态语言缺少编译期类型校验，需要依靠这类工具补齐。大型工程中搭配 mypy 可以减少大量运行时类型错误。

2.8 练习 8

实验内容

```
mkdir -p ex08
```

创建实验目录

```
cd ex08
```

进入工作目录

```
vim metaclass_demo.py
```

编写简单元类，自动校验类属性

```
python3 metaclass_demo.py
```

运行元类代码，观察校验报错

```
vim metaclass_demo.py
```

调整属性约束规则

```
python3 metaclass_demo.py
```

重新运行验证元类拦截逻辑

实验结果 元类在类定义阶段就执行逻辑，不是实例创建时。可以利用元类强制子类必须实现指定方法，完成校验。元编程适合框架开发，业务代码尽量避免复杂元类。元类会增加代码理解成本，新手调试难度较大。框架内部大量使用元类，业务层优先选择装饰器实现。

2.9 练习 9

实验内容

```
mkdir -p ex09
```

创建实验目录

```
cd ex09
```

进入工作目录

```
vim multi_rule.py
```

编写同时存在风格bug和逻辑bug的代码

```
pip install ruff pytest mypy
```

一次性安装全套代码质量工具

```
ruff check multi_rule.py
```

执行代码风格与基础错误扫描

```
mypy multi_rule.py
```

执行静态类型检查

```
python3 -m pytest multi_rule.py
```

执行单元测试验证业务逻辑

实验结果多工具组合构成完整代码质量门禁,各司其职。ruff 负责格式、基础代码缺陷;mypy 负责类型检查。pytest 在运行阶段验证业务逻辑正确性。单一工具只能覆盖部分类型的代码问题。工程上线前执行整套检查,能够大幅降低缺陷数量。

2.10 练习 10

实验内容

```
mkdir -p ex10
```

创建实验目录

```
cd ex10
```

进入工作目录

```
vim code_gen.py
```

编写元编程代码,自动生成Python源码

```
python3 code_gen.py
```

运行脚本,输出生成的代码文件

```
cat generated.py
```

查看自动生成的源码

```
python3 generated.py
```

执行生成出来的代码，验证运行正常

实验结果代码生成是元编程的一种典型应用场景。通过程序自动生成重复模板代码，减少手写重复工作量。自动生成代码可以统一模板规范，避免手写笔误。生成代码可读性差，需要保留生成脚本方便修改维护。序列化、数据模型类代码，经常使用代码生成方式实现。

附录：提交记录

```
yy@yy:~/dev_tools_lab$ git commit -m "docs: 提交第四周实验内容  
- 包含结果文档"  
[master 3ce09d3] docs: 提交第四周实验内容
```

图 25: Enter Caption

```
yy@yy:~/dev_tools_lab/week4$ cd ~/dev_tools_lab  
  
# 移除暂存区里旧的子模块记录  
git rm --cached week4/q16  
  
# 重新把 q16 作为普通文件夹添加  
git add week4/q16/  
  
# 提交修改  
git commit -m "fix: 将q16从子模块恢复为普通实验目录"  
  
# 推送到远程仓库  
git push origin master  
rm 'week4/q16'  
[master 138880d] fix: 将q16从子模块恢复为普通实验目录  
45 files changed, 221 insertions(+), 1 deletion(-)
```

图 26: Enter Caption

GitHub 链接：<https://github.com/123yy-cell/-.git>